

Operating systems

Commands

Content Lecture#4

01	Fork()
02	Exec()
03	File Descriptor
04	Piper()
05	S

Lets take a Break

UNIX is user friendly...

It's just very particular about **who its friends are**.

Why computers are like human:



1. In order to get their attention, you have to turn them on.
2. They have a lot of data, but are still clueless.
3. They are supposed to help you solve problems, but half the time they are the problem.
4. As soon as you commit to one, you realize that if you had waited a little longer, you could have had a better model.

Real programmers count from **0**.

What code does a depressed programmer write?
"Goodbye world!"



File Descriptor

A file descriptor (FD, less frequently fildes) is a process-unique identifier (handle) for a file or other input/output resource

PIPE()

A pipe is a connection between two processes,

Fork()

fork() creates a new process by duplicating the calling process.

SYNOPSIS

```
#include <unistd.h>
pid_t fork(void)
```

The new process is referred to as the *child* process. The calling process is referred to as the *parent* process.

* The child inherits copies of the parent's set of open file descriptors. Each file descriptor in the child refers to the same open file description

Return: On success, the PID of the child process is returned in the parent, and 0 is returned in the child. On failure, -1 is returned in the parent, no child process is created,

```
#include<stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main()
{
printf("Hello\n");
fork();
fork();
fork();
printf("world\n");

wait(NULL);
exit(0);
}
```

Fork commands to give output in Total 2^n and child 2^n-1

Exec()

The **exec**() family of functions replaces the current process image with a new process image.

The **exec**() functions return only if an error has occurred. The return value is -1, and *errno* is set to indicate the error.

SYNOPSIS

```
#include <unistd.h> extern char **environ;

int execl(const char *pathname, const char *arg, ... /*, (char *) NULL */);
int execlp(const char *file, const char *arg, ... /*, (char *) NULL */);
int execle(const char *pathname, const char *arg, ... /*, (char *) NULL, char
*const envp[] */);
int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

File Descriptor

A file descriptor is an unsigned integer used by a process to identify an open file.
The **open**, **pipe**, **creat**, and **fcntl** subroutines all generate file descriptors.
File descriptors are generally unique to each process, but they can be shared by child processes created with a **fork** copied by the **dup**, and **dup2**.
File descriptors are indexes to the file descriptor table in the **u_block** area maintained by the kernel for each process.

A *file descriptor* is the Unix abstraction for an open input/output stream: a file, a network connection, a pipe (a communication channel between processes), a terminal, etc.

When the shell runs a program, it opens three files with file descriptors 0, 1, and 2. The default assignments for these descriptors are as follows

PIPE

pipe() creates a pipe, a unidirectional data channel that can be used for inter-process communication.

On success, zero is returned. On error, -1 is returned, and errno is set appropriately.

SYNOPSIS

```
#include <unistd.h>
int pipe(int pipefd[2]);
```

pipe() creates a pipe, a unidirectional data channel that can be used for interprocess communication. The array pipefd is used to return two file descriptors referring to the ends of the pipe. pipefd[0] refers to the read end of the pipe. pipefd[1] refers to the write end of the pipe. Data written to the write end of the pipe is buffered by the kernel until it is read from

```
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
int
main(int argc, char *argv[])
{
    int pipefd[2];
    pid_t cpid;
    char buf;
    if (argc != 2) {
    fprintf(stderr, "Usage: %s <string>\n", argv[0]);
    exit(EXIT_FAILURE);
    }
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
    cpid = fork();
    if (cpid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    if (cpid == 0) {    /* Child reads from pipe */
        close(pipefd[1]);          /* Close unused write end */
        while (read(pipefd[0], &buf, 1) > 0)
            write(STDOUT_FILENO, &buf, 1);
        write(STDOUT_FILENO, "\n", 1);
        close(pipefd[0]);
        _exit(EXIT_SUCCESS);
    } else {            /* Parent writes argv[1] to pipe */
        close(pipefd[0]);          /* Close unused read end */
        write(pipefd[1], argv[1], strlen(argv[1]));
        close(pipefd[1]);          /* Reader will see EOF */
        wait(NULL);                /* Wait for child */
        exit(EXIT_SUCCESS);
    }
}
```



Thank you